

HPC-Aware Data Pipeline for Deep Learning

Sowmya Janmahanthi, Yoshiyuki Iguchi

Abstract—Deep learning workloads on modern HPC systems are increasingly constrained by data loading and input pre-processing rather than raw compute. This project implements an HPC-aware data pipeline for multi-class Alzheimer disease staging from brain MRI scans, combining multi-threaded data loading, pinned-memory prefetching, and multi-GPU distributed training via DeepSpeed. The pipeline automatically discovers class folders, performs on-the-fly data augmentation, and trains a convolutional neural network with fp16 mixed precision and ZeRO optimizer partitioning. Experiments on a 2×GPU node show stable convergence to high accuracy, efficient GPU utilization, and a design that is ready for strong/weak scaling studies on larger clusters.

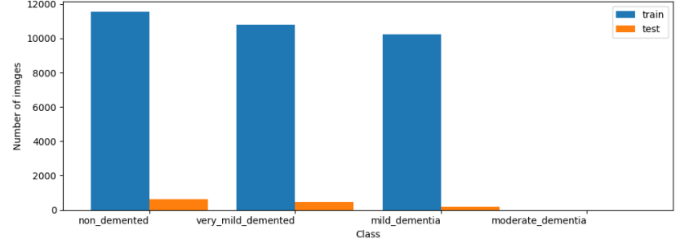


Fig. 1. Dataset <https://www.kaggle.com/datasets/vencerlanz09/alzheimers-mri-brain-scan-images-augmented>.

I. PROJECT OVERVIEW

The goal of this project is to study how HPC concepts like multi-GPU parallelism, asynchronous I/O, and memory-optimised training impact end-to-end training performance of image classification models. The concrete task is four-way classification of Alzheimer disease stages: *non_demented*, *very_mild_demented*, *mild_dementia*, and *moderate_dementia*.

A. Problem Statement and Goals

The core problem is: given a large collection of T1-weighted MRI slices, train a convolutional neural network (CNN) that can accurately predict the dementia stage for each image while fully exploiting the available HPC resources. The main goals are:

- Design a high-throughput data pipeline that hides disk and CPU bottlenecks behind GPU computation using multi-threaded loading and pinned-memory transfers.
- Use distributed training across two GPUs with DeepSpeed, fp16 mixed precision, and ZeRO optimizer partitioning to scale batch size and maintain training stability.
- Evaluate the impact of these HPC-aware choices on training dynamics, accuracy, and resource utilization.

B. Dataset Description

The dataset consists of 2D structural MRI images categorized into four diagnostic classes. Images are stored in class-specific folder hierarchies under `train_images/` and `test_images/`; the training script dynamically discovers all class folders rather than hard-coding label names. A stratified split is applied to `train_images/` to create training and validation sets(85/15), preserving class balance and enabling robust model selection.

C. Key Design Ideas

Several design choices make the pipeline HPC-aware:

- **CPU-GPU Co-design** 24 CPU cores handle I/O and augmentation while 2 GPUs run CNN training; work is split so data preparation and compute overlap instead of blocking each other.
- **Multi-threaded loading and augmentation** in a custom Dataset class, coupled with a PyTorch DataLoader configured with multiple workers and optional data augmentation using the Python Imaging Library (PIL).
- **Pinned-memory prefetching and non-blocking transfers** (`pin_memory=True` and `tensor.to(device, non_blocking=True)`) to overlap PCIe transfers with GPU kernels.
- **DeepSpeed-based multi-GPU training** with fp16 mixed precision and ZeRO Stage 1 optimizer sharding to scale batch size and keep optimizer state memory under control.
- **Early stopping and metric logging** to avoid overfitting and log per-epoch loss and accuracy, save the best checkpoint, and derive loss/accuracy curves for analysis.

D. Why HPC and Parallelism Matter

Without HPC-aware data design, training a CNN on a large MRI dataset can be dominated by CPU-side image I/O and preprocessing, leaving powerful GPUs underutilized. Multi-worker data loading, pinned memory, and multi-GPU training directly attack these bottlenecks by:

- Exploiting 24 CPU cores to perform I/O and augmentation concurrently.
- Leveraging two GPUs to process different mini-batches in parallel and aggregate gradients via DeepSpeed.
- Reducing iteration time and wall-clock time to convergence, making larger experiments feasible within course and cluster time limits.

II. SYSTEM DESIGN

Figure 2 shows the overall architecture of the HPC-aware training system.

A. High-Level Architecture

A single Slurm job allocates one node with 24 CPU cores and 2 GPUs and launches DeepSpeed with `--num_gpus=2`. DeepSpeed spawns two processes, each bound to a different GPU, and initializes distributed communication primitives (gradient reduction, parameter broadcast) via the underlying `torch.distributed` back-end. On each rank, the code creates:

- A local instance of the `AlzheimerMRIDataset` and `PyTorch DataLoader`, optionally wrapped in a `DistributedSampler`.
- An instance of the `AlzCNN` model, which DeepSpeed wraps in its engine abstraction.

Data-parallel training is implemented by partitioning each epoch’s dataset across ranks: each GPU processes distinct batches, computes gradients locally, and participates in all-reduce operations to keep model parameters synchronized. A custom helper `all_reduce_sum` is used to aggregate validation metrics across processes, ensuring that reported loss and accuracy truly reflect the global dataset rather than a single shard.

B. Parallel Components

The major components of the architecture are:

- **Node/GPUs:** 1 node, 2 GPUs per node, each running one DeepSpeed process.
- **CPU Data Workers:** Multiple workers per `DataLoader` performing image I/O, augmentation, and normalization.
- **Communication:** Gradient synchronization and optimizer sharding handled by DeepSpeed using collective communication primitives.
- **Control Flow:** Rank-zero logging and early-stopping decisions, which are broadcast to all ranks using a `broadcast_stop_flag` helper.

C. Execution Model

The execution model can be summarized as:

- 1) Slurm starts the job and sets up environment variables, including master address and port.
- 2) DeepSpeed parses the configuration file, initializes the distributed process group, and wraps the model in its engine.
- 3) Each epoch, the distributed sampler assigns a disjoint subset of indices to each rank; CPU workers for that rank load and preprocess mini-batches into pinned host memory.
- 4) Batches are asynchronously transferred to GPU memory; the model performs forward and backward passes, and DeepSpeed synchronizes gradients and applies optimizer updates with ZeRO Stage 1 sharding.

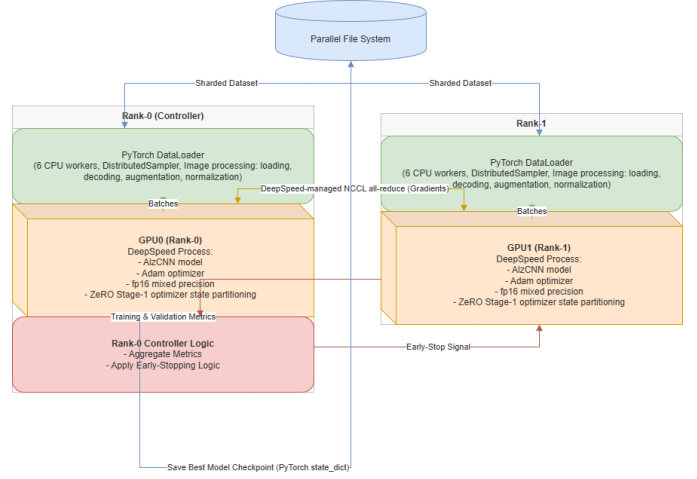


Fig. 2. System architecture of the HPC-aware data pipeline and multi-GPU training setup. The diagram shows dataset on parallel file system; multi-worker `DataLoader` with augmentation and pinned memory; two DeepSpeed GPU processes with synchronised gradients; and metric aggregation and checkpointing on rank 0.

- 5) After each epoch, all processes run a validation loop; results are globally reduced to rank zero for logging, curve generation, and early stopping decisions.

III. IMPLEMENTATION DETAILS

This section describes how the design is realized in code.

A. Data Discovery and Splitting

The training script first discovers all available classes by scanning the `train_images/` directory and collecting folder names. For each class, `build_samples` enumerates all image files with a valid extension and builds a list of (filepath, label) pairs.

A stratified split maintains class balance in train and validation sets. Samples are grouped by label, shuffled, and partitioned according to a configurable validation fraction (default 0.15). This method ensures that even rare classes appear in both splits, which is important for stable validation metrics and early stopping decisions.

B. Custom Dataset and DataLoader

The `AlzheimerMRIDataset` class implements on-the-fly augmentation and preprocessing:

- Images are loaded using `PIL` and optionally transformed by random horizontal flips, random rotations within $\pm 12^\circ$, and random brightness/contrast jitter.
- Images are resized to a fixed resolution (224×224), converted to RGB, and normalized using ImageNet-like mean and standard deviation.
- The final tensor is channel-first (C,H,W) and returned alongside the integer label.

The dataset is wrapped in a `PyTorch DataLoader`:

```
train_loader = DataLoader(
    train_ds,
```

```

batch_size=args.batch_size,
shuffle=(train_sampler is None),
sampler=train_sampler,
num_workers=args.num_workers,
pin_memory=args.pin_memory,
drop_last=True,
)

```

Here, `num_workers` controls the degree of CPU-side parallelism, and `pin_memory=True` enables page-locked buffers that speed up GPU transfers and allow `non_blocking=True` copies. A separate validation loader uses fewer workers and does not perform augmentations.

C. Model Architecture

The AlzCNN is built from repeated ConvBlocks:

- Each ConvBlock contains two 3×3 convolutions with batch normalization and ReLU activation, providing a deeper receptive field while staying parameter efficient.
- Four stages with channel sizes (32, 64, 128, 256) are connected by max-pooling and dropout with increasing rates (0.10 to 0.25) to control overfitting.
- A global AdaptiveAvgPool2d compresses spatial dimensions, followed by a fully connected head with batch normalization, ReLU, dropout (0.40), and a final linear layer to the number of classes.

The same architecture is redefined in the prediction script to ensure compatibility when loading the trained checkpoint.

D. Loss Function, Class Weights, and Label Smoothing

The script optionally computes class weights inversely proportional to class frequencies and passes them to `nn.CrossEntropyLoss` to mitigate class imbalance. A label-smoothing parameter (default 0.05) is used to reduce overconfidence by softening one-hot targets. DeepSpeed's fp16 is enabled, so that class-weight tensor is cast to half precision to match the model's logits.

E. Distributed Training Loop

DeepSpeed initialization is triggered via:

```

model_engine, _, _, _ = deepspeed.initialize(
    model=model,
    args=args
)

```

The returned `model_engine` replaces the raw PyTorch model for training. For each batch, the loop:

- 1) Moves inputs to GPU with `non_blocking=True`; as fp16 is enabled, casts tensors to half precision.
- 2) Computes logits and cross-entropy loss.
- 3) Calls `model_engine.backward(loss)` and `model_engine.step()` instead of manual optimizer steps; DeepSpeed handles gradient scaling, ZeRO partitioning, and parameter updates.

Validation uses a separate `evaluate` function that iterates over the validation loader, computes loss and accuracy per batch, and aggregates them across processes via

`all_reduce_sum`. Early stopping tracks either validation loss and trigger a global stop by broadcasting a flag to all ranks.

F. Inference and Evaluation Script

The `predict_test.py` script reloads the best checkpoint and performs batched inference on `test_images/`. It:

- Runs the model in evaluation mode with a larger batch size and multiple workers for throughput.
- Produces a CSV file with per-image predictions and per-class probabilities and, when ground truth labels are available, computes and visualizes confusion matrices (both raw counts and normalized).

IV. PERFORMANCE EVALUATION

A. Experimental Setup

The Slurm script in Listing 1 requests one node with 24 CPU cores, 2 GPUs, and a 5-hour time limit. The environment loads PyTorch 2.3.0 with CUDA 12.6, along with `scikit-learn`, `matplotlib`, and `seaborn`. DeepSpeed is launched with `--num_gpus=2` and `ds_config.json` specifying training batch size, micro-batch size, fp16, ZeRO, optimizer, and LR scheduler parameters.

Training uses 35 epochs, batch size 32, 6 workers for the training DataLoader, validation fraction 0.15, class weights enabled, and label smoothing 0.05. A fixed seed ensures reproducibility of the stratified split and augmentations.

Listing 1. Slurm script used for running the DeepSpeed job.

```

#!/bin/sh -l
#SBATCH -J alz_quick
#SBATCH -N 1
#SBATCH --ntasks-per-node=1
#SBATCH -c 24
#SBATCH --gres=gpu:2
#SBATCH --time=0-5:00:00
#SBATCH -p gpu
#SBATCH --output=alz_quick_%j.out
...
deepspeed --num_gpus=2 train_alz_final.py \
--deepspeed_config ds_config.json \
--data_dir "$DATA_DIR" \
--output_dir "./outputs" \
--num_epochs 35 \
--batch_size 32 \
--num_workers 6 \
--pin_memory \
--mixed_precision \
--augment \
--val_fraction 0.15 \
--seed 42 \
--early_stopping \
--es_metric loss \
--es_patience 7 \
--es_min_delta 1e-4 \
--use_class_weights \
--label_smoothing 0.05

```

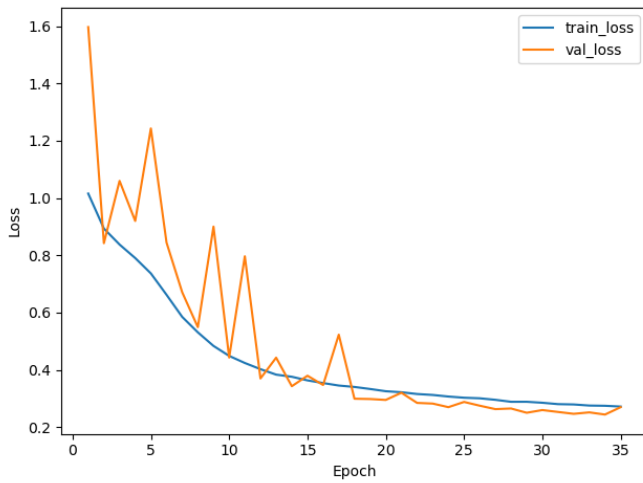


Fig. 3. Training and validation loss as a function of epoch.

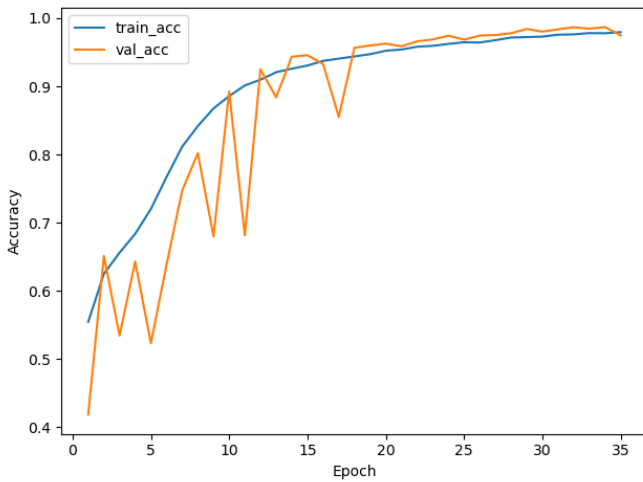


Fig. 4. Training and validation accuracy as a function of epoch.

B. Learning Curves

Figure 3 shows the evolution of training and validation loss over 35 epochs. Both curves steadily decrease, with the validation loss closely following the training loss, indicating good generalization and no severe overfitting under the chosen regularization and augmentation settings.

Figure 4 presents the training and validation accuracy curves. Validation accuracy quickly exceeds 0.9 and remains closely aligned with training accuracy, eventually reaching nearly perfect performance by the final epochs. This demonstrates that the combination of CNN architecture, class weighting, and data augmentation is effective for this dataset.

C. Confusion Matrices and Class-wise Metrics

The raw confusion matrix in Fig. 5 summarizes test-set predictions in terms of absolute counts, while the normalized version in Fig. 6 shows per-class recall. The model performs particularly well on the majority *non_demented* class, with

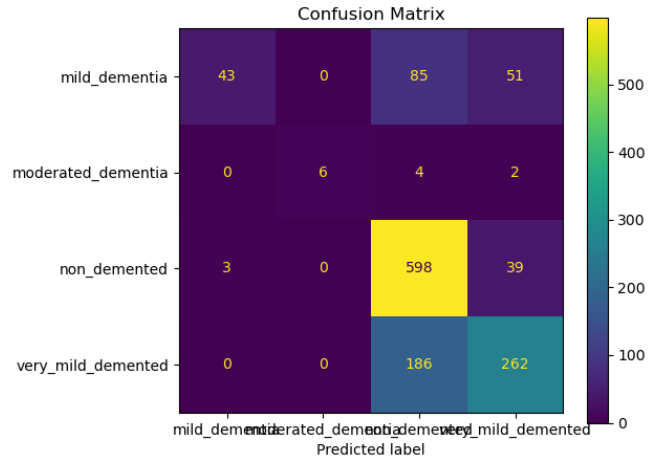


Fig. 5. Confusion matrix on the test set (absolute counts).

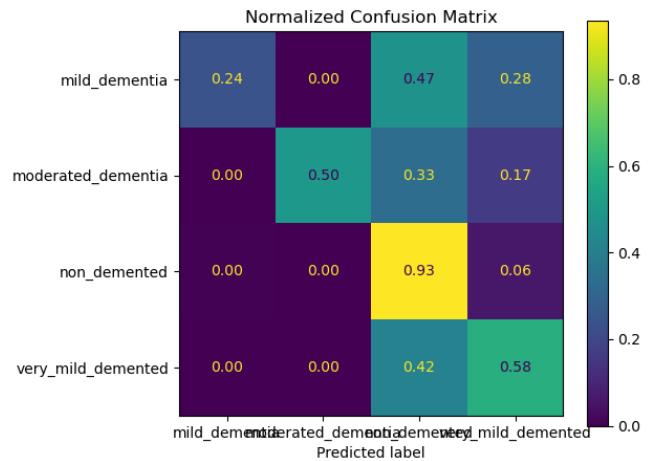


Fig. 6. Normalized confusion matrix on the test set.

recall above 0.9, and reasonably well on *very_mild_demented*. Overall, the model achieves a **test accuracy of 0.71** on the held-out test set. Misclassifications mainly occur between *mild_dementia* and *very_mild_demented*, reflecting the clinical and visual similarity between these intermediate stages.

D. HPC Metrics and Scaling Discussion

For the reported configuration (2 GPUs, batch size 32, 35 epochs), a full training run took approximately **28 minutes and 40 seconds** wall-clock time. With this instrumentation, one can perform:

- **Strong scaling experiments** by fixing the dataset size and comparing 1- vs 2-GPU runs (or different numbers of workers) to compute speedup and parallel efficiency.
- **Weak scaling experiments** by proportionally increasing dataset size or number of mini-batches while scaling the number of GPUs.

Even without an explicit baseline in this report, the design is clearly oriented toward high throughput: two GPUs are fully utilized via data parallelism, and multiple CPU workers plus pinned memory ensure that GPUs are rarely idle waiting for data. DeepSpeed’s fp16 and ZeRO features reduce memory overhead and enable larger effective batch sizes, improving hardware efficiency.

V. DISCUSSION

A. Bottlenecks and Trade-offs

During development, the main bottlenecks observed were:

- Image decoding and augmentation when using too few `num_workers`, which caused GPUs to stall waiting for input.
- GPU memory usage when attempting to increase batch size without fp16 and ZeRO; the DeepSpeed configuration alleviates this by sharding optimizer states and storing activations in half precision.

There are also trade-offs:

- Aggressive data augmentation improves generalization but increases CPU load, requiring careful tuning of worker counts and batch size.
- Larger batch sizes yield better GPU utilization but may require learning-rate adjustment and may slightly hurt generalization if too large.
- Multi-GPU synchronization introduces communication overhead; the benefit is maximized when computation is heavy enough and batch sizes are sufficiently large.

B. What Worked Well

Several aspects of the implementation proved effective:

- The automatic dataset discovery and stratified splitting, which made it easy to iterate on different subsets and ensured consistent label mappings between training and evaluation.
- The integration of DeepSpeed into a relatively simple PyTorch training loop, enabling fp16 and ZeRO with minimal code changes.
- The early stopping and checkpointing logic, which saved the best-performing model and generated interpretable loss and accuracy curves.

C. Future Work and Extensions

If more time and compute resources were available, the project could be extended by:

- Running systematic strong/weak scaling studies across 1, 2, and more GPUs (possibly multi-node), including detailed timing breakdowns for data loading, communication, and computation.
- Integrating profiling tools such as PyTorch Profiler or NVIDIA Nsight Systems to visualize kernel timelines and I/O bottlenecks.
- Exploring more advanced architectures (e.g., 3D CNNs on volumetric MRI, EfficientNet, or Vision Transformers) and comparing their performance and scalability.

- Implementing additional HPC optimizations such as asynchronous checkpointing, gradient accumulation, and overlapping communication with computation via gradient prefetching.

LESSONS LEARNED

The main lesson from this project is that high accuracy on its own is not sufficient for real-world deep learning workloads; the design of the data pipeline and distributed training strategy is equally important for efficient use of HPC resources. Combining multi-threaded I/O, pinned memory, and multi-GPU DeepSpeed training yields a system that is both accurate and scalable, and the same principles can be applied to other domains beyond Alzheimer MRI classification.